JAVA

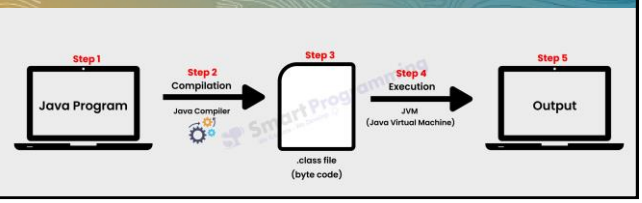
DESENVOLVIMENTO DE APLICAÇÕES

Prof. Leonardo Coelho | 2026

TEMA 05







1

DESENVOLVIMENTO DE APLICAÇÕES

ORIENTAÇÃO A OBJETOS - POO

O Java foi totalmente construído em cima desse conceito.

Entender **Orientação a Objetos** é o que diferencia um iniciante de um programador profissional.

O que é a Orientação a Objetos?
Até agora, nós criamos variáveis soltas (uma String nome aqui, um int idade ali). Na vida real, as coisas não andam soltas, elas andam juntas em um "**pacote**".



2

DESENVOLVIMENTO DE APLICAÇÕES

ORIENTAÇÃO A OBJETOS - POO

Pense em um Smartphone.

Ele não é apenas uma cor isolada ou um número de bateria isolado. Um smartphone é um Objeto que junta duas coisas:

Características (Atributos): Marca, Modelo, Cor, Capacidade de Armazenamento.

Ações (Métodos): Tirar Foto, Ligar, Instalar Aplicativo.A Classe em Java funciona exatamente como o molde ou a planta de engenharia para fabricar esses objetos.



3

DESENVOLVIMENTO DE APLICAÇÕES

CÓDIGO PRÁTICO (Como criar e usar uma CLASSE)

Arquivo 1: Celular.java

Este arquivo é apenas o molde. Repare que ele não tem aquele método main. Ele apenas define o que um celular tem e o que um celular faz:

```
public class Celular {

   String marca;
   String modelo;
   int bateria;

   public void tirarFoto() {
      System.out.println("Clic! Foto tirada com sucesso.");
   }
}
```

4

DESENVOLVIMENTO DE APLICAÇÕES

CÓDIGO PRÁTICO (Como criar e usar uma CLASSE)

Arquivo 2: Main.java

Este é o arquivo principal que você já conhece. É aqui que vamos usar o molde Celular para fabricar um celular de verdade na memória do computador.

```
public class Main {

    public static void main(String[] args) {
        Celular meuCelular = new Celular();

        meuCelular.marca = "Apple";
        meuCelular.modelo = "iPhone 15";
        meuCelular.bateria = 100;

        System.out.println("Meu celular é um " +
meuCelular.modelo + " da " + meuCelular.marca);

        meuCelular.tirarFoto();
    }
}
```

5

DESENVOLVIMENTO DE APLICAÇÕES

CÓDIGO PRÁTICO (Como criar e usar uma CLASSE) | EXEMPLO

O que acabou de acontecer aqui?

Celular meuCelular = new Celular();
A palavra-chave **new** é mágica. Ela diz ao Java: "Vá até o molde Celular e crie um objeto real na memória chamado meuCelular".

O uso do Ponto (.): Usamos o ponto para acessar o que está dentro do objeto, seja para dar uma característica **(meuCelular.marca = "Apple")** ou para fazê-lo realizar uma ação **(meuCelular.tirarFoto()).**

6

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Atividade: Sistema Bancário (Estrutura POO)

Para estruturar um sistema bancário em Java utilizando os pilares da Programação Orientada a Objetos (POO), o ideal é separar as responsabilidades em classes distintas.

Isso torna o código limpo, modular e fácil de manter.

Como estamos focando na lógica estrutural, o ideal e desenvolver uma modelagem utilizando classes conceituais e uma implementação prática via console (Terminal) para simular o fluxo que você descreveu.



7

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

Para estruturar um sistema bancário em Java utilizando os pilares da Programação Orientada a Objetos (POO), **o ideal é separar as responsabilidades em classes distintas.**

Isso torna o código limpo, modular e fácil de manter.

Como estamos focando na lógica estrutural, o ideal e desenvolver uma modelagem utilizando classes conceituais e uma implementação prática via console (Terminal) para simular o fluxo que você descreveu.



8

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

🏛 Modelagem das Classes (Estrutura POO)

Para esse sistema, podemos definir 3 classes principais:

1. `ContaBancaria` : Representa a conta do usuário, guardando os dados (número, saldo, senha) e os métodos de movimentação (depositar, sacar).
2. `Banco` : Gerencia o banco de dados (simulado em memória), controlando a criação de contas e a validação de login.
3. `SistemaTerminal` : A classe principal que lida com a interface do usuário (menus, entradas do teclado e loops de navegação).

9

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

1. Classe ContaBancaria (Entidade)

💻 Implementação do Código

1. Classe `ContaBancaria` (Entidade)

Esta classe encapsula os dados da conta. Reparou no uso de `private` ? Isso garante o **encapsulamento**, permitindo que o saldo só seja alterado por métodos validados (`sacar` e `depositar`).

10

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

1. Classe ContaBancaria (Entidade)

```java
public class ContaBancaria {
    private String numeroConta;
    private String senha;
    private double saldo;

    // Construtor
    public ContaBancaria(String numeroConta, String senha, double saldoInicial) {
        this.numeroConta = numeroConta;
        this.senha = senha;
        this.saldo = saldoInicial;
    }

    // Métodos específicos da regra de negócio
    public boolean verificarSenha(String senhaDigitada) {
        return this.senha.equals(senhaDigitada);
    }
```

11

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

1. Classe ContaBancaria (Entidade)

```java
public void depositar(double valor) {
    if (valor > 0) {
        this.saldo += valor;
        System.out.println("Depósito de R$ " + valor + " realizado com sucesso!");
    } else {
        System.out.println("Valor de depósito inválido.");
    }
}
```

12

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

1. Classe ContaBancaria (Entidade)

```
    public boolean sacar(double valor) {
        if (valor > 0 && valor <= this.saldo) {
            this.saldo -= valor;
            System.out.println("Saque de R$ " + valor + " realizado com sucesso!");
            return true;
        } else {
            System.out.println("Saldo insuficiente ou valor inválido.");
            return false;
        }
    }
```

13

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

1. Classe ContaBancaria (Entidade)

```
    // Getters
    public String getNumeroConta() {
        return numeroConta;
    }

    public double getSaldo() {
        return saldo;
    }
}
```

14

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

2. Classe Banco (Gerenciamento)

Responsável por armazenar as contas e realizar a autenticação.

```java
import java.util.ArrayList;
import java.util.List;

public class Banco {
    private List<ContaBancaria> contas;

    public Banco() {
        this.contas = new ArrayList<>();
        // Inicializando algumas contas para teste
        contas.add(new ContaBancaria("1234", "1111", 500.00));
        contas.add(new ContaBancaria("5678", "2222", 1500.00));
    }
```

15

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

2. Classe Banco (Gerenciamento)

Responsável por armazenar as contas e realizar a autenticação.

```java
// Método para autenticar usuário
public ContaBancaria fazerLogin(String numeroConta, String senha) {
    for (ContaBancaria conta : contas) {
        if (conta.getNumeroConta().equals(numeroConta) && conta.verificarSenha(senha)) {
            return conta; // Retorna a conta autenticada
        }
    }
    return null; // Credenciais inválidas
}
}
```

16

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

Aqui gerenciamos o loop principal do sistema. Note o uso de dois loops **while:** o externo controla a tela de login e o interno controla as ações do usuário logado. Quando ele escolhe "Sair", o loop interno quebra e ele volta automaticamente para o login.

```
import java.util.Scanner;

public class SistemaTerminal {
    public static void main(String[] args) {
        Banco banco = new Banco();
        Scanner scanner = new Scanner(System.array);
        boolean rodandoSistema = true;
```

17

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

```
        // Loop Principal do Sistema (Tela de Login)
        while (rodandoSistema) {
            System.out.println("\n--- BEM-VINDO AO BANCO ---");
            System.out.print("Digite o número da conta (ou 'sair' para fechar o app)
            String contaInput = scanner.nextLine();

            if (contaInput.equalsIgnoreCase("sair")) {
                rodandoSistema = false;
                System.out.println("Sistema encerrado. Até logo!");
                break;
            }

            System.out.print("Digite a senha: ");
            String senhaInput = scanner.nextLine();
```

18

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

```
// Tentativa de login
ContaBancaria contaLogada = banco.fazerLogin(contaInput, senhaInput);

if (contaLogada != null) {
   System.out.println("\nLogin realizado com sucesso!");
   boolean logado = true;

   // Loop da Conta Logada (Menu de Operações)
   while (logado) {
      System.out.println("\n--- MENU DA CONTA: " + contaLogada.getNumeroConta() + " ---");
      System.out.println("1. Verificar Saldo");
      System.out.println("2. Realizar Depósito");
      System.out.println("3. Realizar Retirada (Saque)");
      System.out.println("4. Sair (Desconectar)");
      System.out.print("Escolha uma opção: ");
```

19

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

```
int opcao = scanner.nextInt();
scanner.nextLine(); // Limpar buffer do teclado

switch (opcao) {
   case 1:
      System.out.println("Seu saldo atual é: R$ " + contaLogada.getSaldo());
      break;
   case 2:
      System.out.print("Digite o valor do depósito: ");
      double valorDeposito = scanner.nextDouble();
      contaLogada.depositar(valorDeposito);
      break;
```

20

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

```
                case 3:
                    System.out.print("Digite o valor da retirada: ");
                    double valorRetirada = scanner.nextDouble();
                    contaLogada.sacar(valorRetirada);
                    break;
                case 4:
                    System.out.println("Desconectando da conta...");
                    logado = false; // Quebra este loop e volta para o menu de login
                    break;
                default:
                    System.out.println("Opção inválida!");
            }
        }
```

21

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

3. Classe SistemaTerminal (Interface e Fluxo)

```
} else {
            System.out.println("Conta ou senha incorretos. Tente novamente.");
        }
    }
    scanner.close();
  }
}
```

22

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Sistema Bancário (Estrutura POO)

🛠 Pilares de POO aplicados aqui:

- **Encapsulamento**: Os atributos como `saldo` e `senha` são privados (`private`). O mundo externo não pode alterá-los diretamente, apenas através dos métodos públicos como `sacar()` e `depositar()` , protegendo as regras de negócio.
- **Abstração**: Criamos um modelo digital (`ContaBancaria`) focado apenas nas propriedades necessárias para o sistema (saldo, número, senha), ignorando detalhes irrelevantes do mundo real.
- **Reutilização e Modularidade**: Se amanhã você quiser trocar a interface de console por uma tela gráfica (JavaFX ou Web), as classes `Banco` e `ContaBancaria` permanecem exatamente iguais, pois a lógica de negócios está isolada da interface.

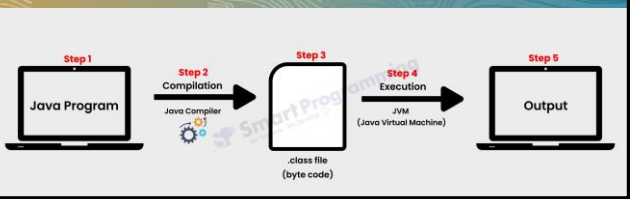
23

JAVA

DESENVOLVIMENTO DE APLICAÇÕES







24